

Atelier Matplotlib

Contexte

Une entreprise possède plusieurs bâtiments équipés de capteurs IoT.

Chaque capteur collecte régulièrement des informations sur la température, l'humidité, la pression, la consommation énergétique, l'état du capteur, le bâtiment, la date et l'heure de la mesure.

Après avoir utilisé **NumPy** pour manipuler les données numériques et **Pandas** pour importer, nettoyer et analyser le dataset, l'étape suivante consiste à **représenter graphiquement les informations afin d'identifier des tendances, des différences entre bâtiments et des anomalies**.

Structure du projet

- 1) Structurer le projet comme suit :

```
atelier_matplotlib_iot/  
├── data/  
│   └── mesures_capteurs.csv  
├── notebooks/  
│   └── atelier_matplotlib_iot.ipynb  
└── exports/  
    ├── temperature.png  
    └── temperature.pdf
```

- 2) Stocker le fichier fourni **mesures_capteurs.csv** dans le sous-dossier **data**
- 3) Créer le notebook **atelier_matplotlib_iot.ipynb**
- 4) Installer et importer **matplotlib** et **pandas**
- 5) Importer **mesures_capteurs.csv** dans le dataframe **df**

Partie 1 – Graphique linéaire (Line Plot)

- 1) Afficher l'évolution de la température en fonction du temps, avec le titre, les labels, la légende et la grille.
- 2) Identifier visuellement les valeurs qui semblent anormales.

Partie 2 – Diagramme en barres (Bar Chart)

- 1) Déterminer **consommation_batiment** qui stocke les consommations moyennes des bâtiments
- 2) Comparer graphiquement la consommation moyenne des bâtiments
- 3) Comparer la consommation moyenne des bâtiments avec un graphique horizontal
- 4) Dans quels cas un graphique horizontal peut-il être plus lisible ?

Partie 3 – Histogramme

- 1) Afficher la distribution des températures en divisant la plage des données en 20 classes et en affichant uniquement les lignes horizontales de la grille
- 2) Analyser le graphique pour voir

- a) où se concentrent les températures ;
 - b) si la distribution semble symétrique ;
 - c) s'il existe des valeurs très éloignées ;
 - d) si des valeurs aberrantes sont visibles.
- 3) Afficher la distribution de la consommation en divisant la plage des données en 20 classes
 - a) Modifier le nombre de classes à 10
 - b) Modifier le nombre de classes à 30
 - c) Quel impact le nombre de classes a-t-il sur l'interprétation ?

Partie 4 – Nuage de points (Scatter Plot)

- 1) Réaliser un nuage de points entre température et consommation
- 2) Déterminer visuellement s'il semble exister une relation entre les deux

Partie 5 – Diagramme à moustache (Box plot)

- 1) Créer un box plot de la température. N'oublier pas de supprimer les valeurs manquantes avant de construire le graphique et afficher uniquement les lignes horizontales de la grille
- 2) Que représentent les points situés en dehors des moustaches ?
- 3) Créer un box plot de la consommation
- 4) A partir des deux box plot, comparer :
 - a) la position de la médiane ;
 - b) la taille de la boîte ;
 - c) la longueur des moustaches ;
 - d) les valeurs extrêmes.
- 5) La consommation présente-t-elle davantage de dispersion que la température ?

Partie 6 – Diagramme circulaire (Pie Chart)

- 1) Réaliser un diagramme pour analyser la répartition des états (OK, ALERTE, ERREUR)
- 2) interpréter la proportion de capteurs OK ; proportion ALERTE ; proportion ERREUR

Partie 7 : Plusieurs courbes sur un même graphique

Construire sur un même graphique « Évolution de la température par bâtiment" » des quatre bâtiments.

Partie 8 : Sauvegarde des graphiques

- 1) Sauvegarder le graphique sur l'évolution de la température en fonction du temps dans **exports/ temperature.png**
- 2) Sauvegarder le graphique sur l'évolution de la température en fonction du temps dans **exports/ temperature.pdf**

Partie 9 : Bonus

Proposer et réaliser toute fonctionnalité pertinente et utile qu'on pourrait rajouter à l'atelier.

Livrable attendu

Chaque apprenant devra fournir le dossier **atelier_matplotlib_iot**

Modalité de livraison

Chaque apprenant pousse

- 1) le livrable, **d'abord vide**, sur un dépôt public de son compte Github personnel
- 2) met à jour le livrable, au fur et à mesure, avec des messages de commit explicites